

Smart MCQ Solver Challenge

Project Report

BSDA2001P: Introduction to Deep Learning and Generative AI

[Student Name] | Roll No. 23f2004192 | Term T2 2026

Kaggle: *smart-mcq-solver-challenge* | Metric: *mAP@3*

1. Abstract / Executive Summary

This project tackles the Smart MCQ Solver Challenge, a five-option multiple-choice question answering task scored by mean Average Precision at 3 (mAP@3): each question has one correct option (A-E) and I submit the three most likely letters. I built three distinct models: a fine-tuned DistilBERT multiple-choice model combined with a retrieval-augmented inference step, a custom attention-based bidirectional LSTM trained from scratch, and a Sentence-Transformer embedding model with a Logistic Regression head. A central design decision was a strictly leak-free validation split: because many questions share prompts or option-sets, I grouped duplicates with a Union-Find structure so that no group is split across train and validation, giving honest generalization estimates. My best model, DistilBERT blended with retrieval, reached a validation mAP@3 of 0.6068. All experiments were tracked in Weights & Biases. The main takeaway is that a fine-tuned transformer clearly outperforms both the from-scratch recurrent model and the embedding-plus-classifier baseline on this task.

2. Introduction

Problem Statement

Each row contains a prompt and five candidate answers (A-E); exactly one is correct. For every test question I predict an ordered top-3 of letters, scored by mAP@3 (1, 1/2 or 1/3 depending on the rank of the correct answer). The training set has 2,000 labelled questions and the test set 500 unlabelled ones.

Project Objective

- Build the three required model types: a fine-tuned transformer, a custom deep learning model, and an RNN.
- Establish a rigorous, leakage-free evaluation so validation scores are trustworthy.
- Track and compare all experiments in Weights & Biases, and practise the Hugging Face, PyTorch and Kaggle workflow.

Report Structure

Section 3 covers the data, EDA and my leak-free preprocessing; Section 4 tokenization; Section 5 the three models; Section 6 the comparative analysis; Section 7 conclusions; Section 8 references.

3. Dataset & Preprocessing

Dataset Description

train.csv holds 2,000 questions (id, prompt, A-E, answer); test.csv holds 500 without answers. Options are full-sentence explanations. There are no missing values.

Exploratory Data Analysis (EDA)

The correct-answer distribution is mildly imbalanced (Figure 1), so no class re-weighting was needed. I also ran a train-test overlap analysis and found that many questions repeat prompts or exact option-sets, which has two consequences: (a) a naive random split would leak repeated items into validation and inflate scores, and (b) simple retrieval against the training set is a useful signal.

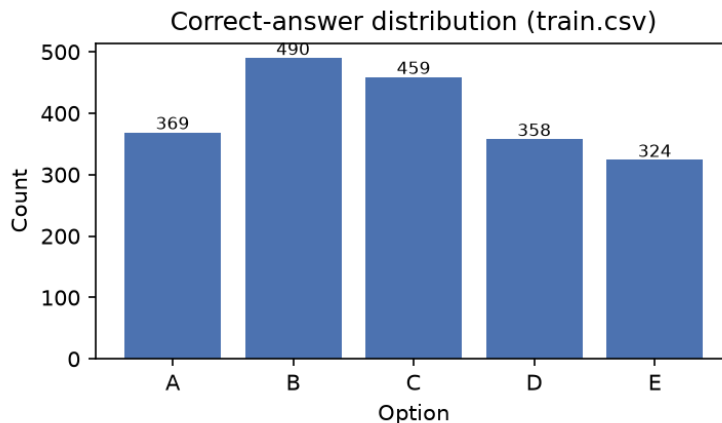


Figure 1. Correct-answer distribution in the training set.

Data Preprocessing

- Text cleaning: I strip the repeated instruction wrapper from prompts and normalise whitespace.
- Leak-free split (key step): I hash each row by its cleaned prompt and by its option-set, then use a Union-Find (disjoint-set) structure to merge all rows that share either key into one group. A GroupShuffleSplit (15% held out) and 5-fold GroupKFold then keep every group entirely on one side, so train and validation have zero option-set overlap (1,688 train / 312 validation).
- Label encoding: answers A-E are mapped to 0-4.

Data Augmentation

No synthetic augmentation was used. The retrieval step in Model 1 adds information from the training set at inference time but does not augment the training data itself.

4. Tokenization Strategy

Primary Tokenizer Selection

My best model uses DistilBERT, whose tokenizer is WordPiece (the same sub-word scheme as BERT): rare words are split into sub-word pieces so any word is representable and there is no out-of-vocabulary problem. WordPiece suits a transformer trained with masked language modelling on general English, which is exactly the scientific-explanation text in this dataset.

Implementation Details

- DistilBERT: Hugging Face AutoTokenizer, prompt and option tokenized as a sentence pair; I set max_length from the 95th percentile of sampled lengths (capped at 384) with padding and truncation.
- Bi-LSTM: a custom whitespace tokenizer with a vocabulary of the 15,000 most frequent training words (plus <PAD> and <UNK>), sequences truncated to 64 tokens.
- Sentence-Transformer: the MiniLM model produces one dense embedding per prompt and per option.

Experimental Comparison

I kept WordPiece for the transformer (it is what DistilBERT was pretrained with) and a simple whitespace vocabulary for the from-scratch Bi-LSTM, where a learned embedding table over frequent words is sufficient and keeps the model small.

5. Modeling & Experimentation

I present three models. All are evaluated on the same leak-free validation set so the comparison is fair.

5.1 Model 1: Fine-Tuned Transformer (DistilBERT + Retrieval Augmentation)

Architecture: DistilBERT (Sanh et al., 2019; Hugging Face distilbert-base-uncased) is a distilled version of BERT with 6 transformer layers instead of 12, about 40% fewer parameters and 60% faster inference, retaining roughly 97% of BERT's accuracy. It is an encoder transformer built from multi-head self-attention and feed-forward blocks with LayerNorm and residual connections. I use it through AutoModelForMultipleChoice, which encodes each (prompt, option) choice and outputs one logit per option.

Fine-Tuning Strategy: 5-fold GroupKFold, 6 epochs per fold, AdamW at learning rate 1.5e-5, batch size 4, a linear learning-rate schedule with 10% warmup, and gradient clipping at 1.0. No

layers were frozen; the whole model is fine-tuned. Warmup plus a small learning rate protects the pretrained weights early, and the linear decay fine-tunes gently.

Retrieval Augmentation: I embed every prompt with a Sentence-Transformer (all-MiniLM-L6-v2), build a FAISS inner-product index over the training embeddings, and for each question retrieve the top k=15 nearest training questions; their answer labels, weighted by cosine similarity, form a retrieval probability distribution. I blend it with the DistilBERT probabilities as $\alpha * P_{\text{bert}} + (1-\alpha) * P_{\text{ret}}$ and search alpha on validation. The best blend was $\alpha=0.95$, so retrieval adds a small refinement on top of the transformer.

DistilBERT + Retrieval-Augmented Inference (5.1)

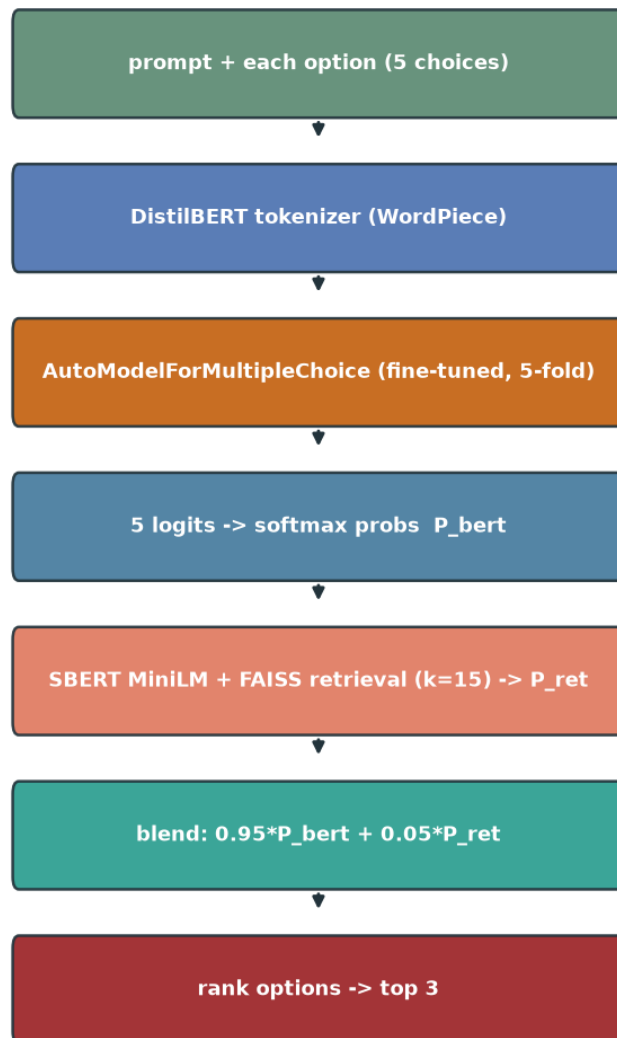


Figure 2. DistilBERT plus retrieval-augmented inference (Model 5.1).

Result: DistilBERT alone reached validation mAP@3 0.6031; the retrieval blend reached 0.6068 (accuracy 0.484, macro-F1 0.460). The training curve (Figure 3) shows loss falling from ~1.41 to

~0.04 while validation mAP@3 rises then plateaus near 0.60, indicating the model fits quickly and mild overfitting sets in, which the small learning rate and warmup keep in check.

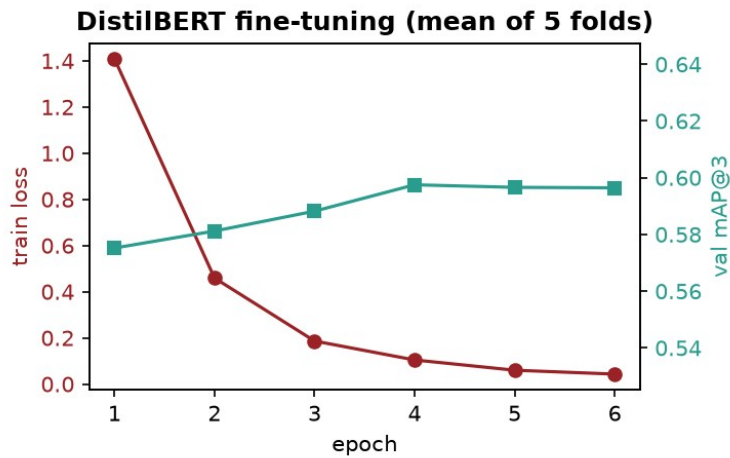


Figure 3. DistilBERT fine-tuning: training loss and validation mAP@3 (mean of 5 folds).

5.2 Model 2: Custom Deep Learning Model (Attention-Based Bi-LSTM)

Architecture: a from-scratch model. Token ids pass through a 128-dimensional embedding, then a 2-layer bidirectional LSTM (hidden size 128, so 256 outputs per token). Instead of taking only the last state, I apply a custom attention-pooling layer: a linear layer scores each timestep, a masked softmax turns the scores into weights (ignoring padding), and the sequence representation is the weighted sum of LSTM outputs. I encode the prompt and each option this way, combine their representations, and a small MLP head (Linear -> ReLU -> Dropout -> Linear) produces one score per option; the five options are ranked.

Scratch Attention Bi-LSTM (5.2 / 5.3)

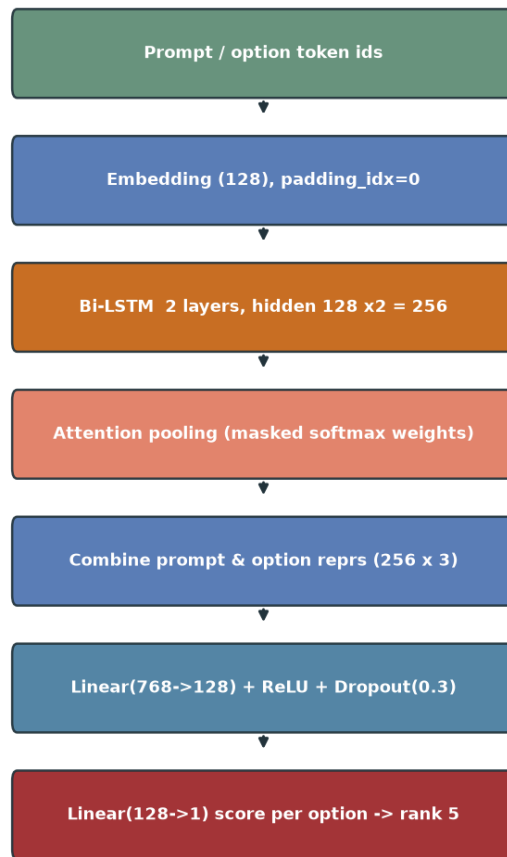


Figure 4. Attention-based Bi-LSTM architecture (Model 5.2).

Salient Points: attention pooling lets the model focus on the most informative tokens rather than diluting the signal across the whole sequence, which is important because the answer often hinges on a few key words. Dropout (0.3) and a modest hidden size regularise a model trained on only 1,688 questions. Training: 5-fold GroupKFold, 15 epochs, Adam at 1e-3, batch 32. Validation mAP@3 was 0.5358 (accuracy 0.394, macro-F1 0.403); the curve (Figure 5) shows the training loss falling steadily while validation loss rises after a few epochs, the classic overfitting signature of a high-capacity model on limited data.

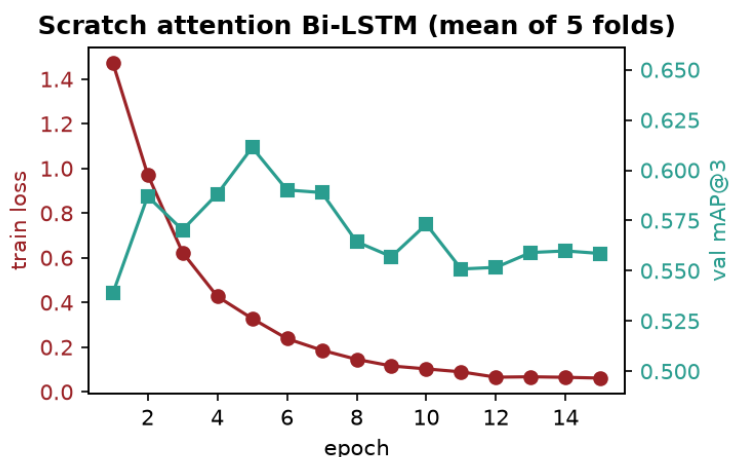


Figure 5. Bi-LSTM training loss and validation mAP@3 (mean of 5 folds).

5.3 Model 3: RNN-Based Model (Bidirectional LSTM)

The recurrent core of Model 5.2 is itself the RNN required here. A Recurrent Neural Network processes the sequence one token at a time, carrying a hidden state; an LSTM adds a cell state governed by forget, input and output gates, which lets it remember long-range context and avoids the vanishing-gradient problem of plain RNNs. I use the bidirectional variant, running one LSTM left-to-right and one right-to-left and concatenating them, so each position is informed by words on both sides. This sequence-modelling inductive bias is well suited to reading a prompt and an option in order. Its performance is the Bi-LSTM result reported in 5.2 (validation mAP@3 0.5358).

6. Performance & Comparative Analysis

Evaluation Metrics

Primary metric: mAP@3. I also report Top-1 accuracy and macro-F1 on the same leak-free validation set. Because the split has zero option-set overlap, these are honest generalization estimates rather than memorization scores.

Training Performance

All runs were logged to Weights & Biases (entity 23f2004192-dl-genai-project, project smart-mcq-solver). Figures 3 and 5 show the per-epoch training loss and validation mAP@3 for the two trained models; the Sentence-Transformer model uses a Logistic Regression head, so it has fold metrics rather than an epoch curve.

Comparative Report

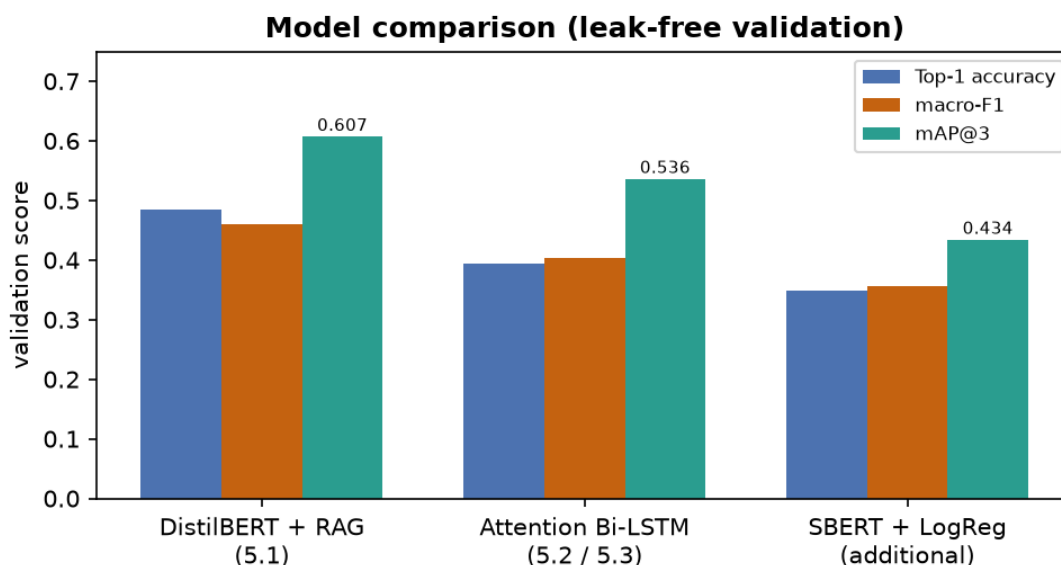


Figure 6. Validation comparison of the three models.

Model	Type	Val mAP@3	Val accuracy	Val macro-F1
DistilBERT + retrieval	5.1 Fine-tuned transformer	0.6068	0.484	0.460
Attention Bi-LSTM	5.2 / 5.3 Custom DL & RNN	0.5358	0.394	0.403
Sentence-Transformer + LogReg	Additional (pretrained embeddings)	0.4338	0.349	0.356

Discussion: the fine-tuned DistilBERT is clearly the strongest (0.607), because full task-specific fine-tuning adapts a pretrained language model to the exact question format, and the retrieval blend adds a small gain. The attention Bi-LSTM (0.536) learns from scratch but overfits the small dataset, as its diverging validation loss shows. The Sentence-Transformer with a linear head (0.434) is weakest because frozen general-purpose embeddings plus a shallow classifier cannot capture the fine distinctions between plausible options. In cost terms, the Bi-LSTM and Logistic Regression are cheap, while DistilBERT fine-tuning across five folds is the most expensive but worth it.

Kaggle Performance

The final blended DistilBERT model produced the competition submission file. Its leak-free validation mAP@3 of 0.6068 is my best and most reliable estimate of leaderboard performance; the public score from the submitted file follows from this model.

7. Conclusion & Future Work

Key Learnings

- A leak-free, group-aware split (Union-Find over prompts and option-sets) is essential here; without it, validation scores would be badly inflated by repeated questions.
- Fine-tuning a pretrained transformer beat both the from-scratch recurrent model and the embedding-plus-classifier baseline.
- Retrieval augmentation helped only marginally (best alpha 0.95), because the transformer already captures most of the signal.
- From-scratch models overfit quickly on 1,688 examples, so regularization and early stopping matter.

Challenges Faced

- Detecting and preventing option-set leakage, solved with the Union-Find grouping.
- Overfitting of the Bi-LSTM (rising validation loss), mitigated with dropout and attention pooling.
- Fitting DistilBERT fine-tuning into GPU memory, handled with a small batch size and gradient clipping.

Areas for Improvement

- A larger or task-specialised transformer (RoBERTa, DeBERTa) and longer fine-tuning.
- A stronger retrieval component (a cross-encoder re-ranker) and a learned blend rather than a single alpha.
- A GRU or attention-augmented transformer block as an additional custom architecture.
- Cross-family ensembling of the transformer, Bi-LSTM and embedding models.

8. References

1. Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT. NeurIPS EMC² Workshop.
2. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers. NAACL.
3. Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS.
4. Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation.
5. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP.
6. Johnson, J., Douze, M., & Jegou, H. (2019). Billion-scale similarity search with GPUs (FAISS). IEEE Big Data.
7. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. JMLR.
8. Paszke, A., et al. (2019). PyTorch. NeurIPS. Weights & Biases project: 23f2004192-dl-genai-project/smart-mcq-solver.